



Stage M2 en Entreprise

Développement d'une application d'incrustation visuelle d'équipements thermiques et de climatisations par Intelligence Artificielle

Soitel Rémi



UNIVERSITÉ
DE REIMS
CHAMPAGNE-ARDENNE

Sommaire

- Introduction
- Contexte, Problématique et Objectif
- Etude et Choix
- Implémentation
- Perspective d'Amélioration
- Conclusion

Remerciement

- Je tiens à remercier Monsieur Stéphane SOITEL, Président de Chauffage Der Energies et tuteur de stage, pour son encadrement, ses remarques et son soutien tout au long du stage.
- Merci également à Monsieur Hacène Fouchal, Responsable de formation du Master Informatique, pour l'opportunité d'effectuer ce stage et d'y appliquer nos connaissances.
- Enfin, merci à l'ensemble de l'équipe de Chauffage Der Energies pour leur accueil chaleureux et l'environnement de travail propice à l'apprentissage

Introduction

- Dans le cadre de mes études, j'ai intégré l'entreprise Chauffage Der Energies pour un stage de 4 mois afin d'y appliquer mes connaissances.
- L'entreprise est spécialisée dans l'installation de solution thermiques et climatiques dans lequel se présente un défi commercial :
 - *La projection du client de l'installation souhaiter.*
- Pour palier à ça, l'entreprise m'a offert comme mission de développer une application d'incrustation réaliste d'équipement thermique et climatique.



Contexte et Problématique

- L'entreprise travaille sur un marché concurrentiel où la technique ne suffit plus pour vendre :
 - Les catalogues papiers et photos ne permettent pas la projection du client.
 - Il est difficile d'imaginer l'encombrement de l'appareil et le passage des tuyaux.
 - Le client peut avoir peur d'une « dégradation » esthétique.
- C'est pour palier à tout ça que l'idée d'une application qui simule la présence d'un équipement dans une pièce pourrait aider les clients à se rassurer.

Objectif et Contrainte du Projet

- Pour que l'application se construise, on s'est fixé plusieurs objectifs :
 - Assurer un photoréalisme au top avec un respect :
 - De la perspective
 - Des ombres sur l'équipement et de la pièce
 - De l'échelle réelle de l'équipement
 - De la colorimétrie de la pièce
 - Suivre des contraintes techniques :
 - L'application doit tourner en local
 - Doit faire les traitements rapidement



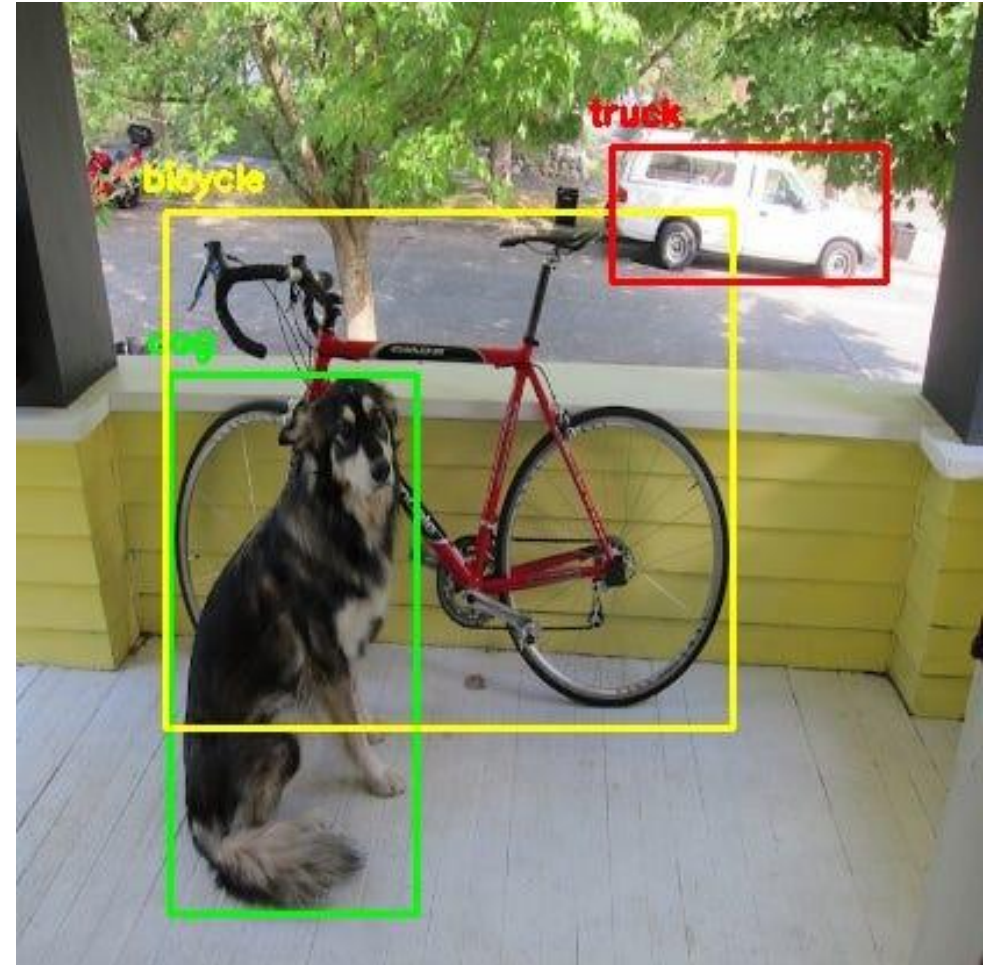
Etude et Choix - Autocollant

- Pour avoir une échelle réelle, il nous fallait un repère absolu dans l'espace.
- On a choisi d'utiliser un autocollant vert de 10cm par 5cm :
 - Contraste avec la majorité des couleurs de murs (peinture, brique, crépi).
 - Très visible pour les capteurs photo et pour la reconnaissance par IA.
 - Sert d'ancrage absolu pour calculer la taille réelle des équipements en pixels.



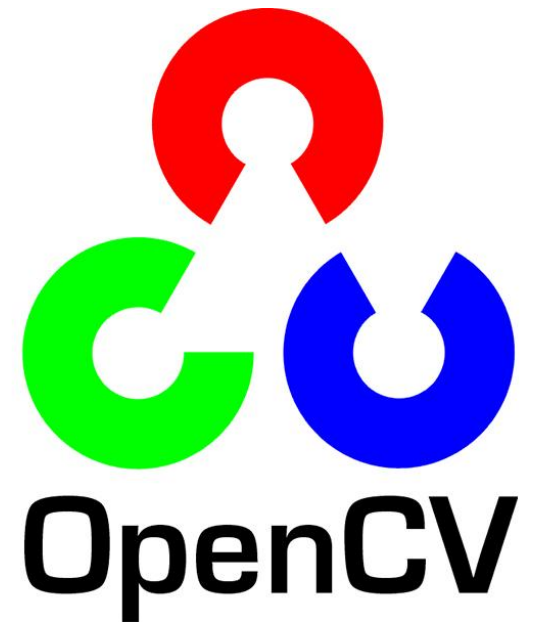
Etude et Choix - Intelligence Artificielle

- Reconnaissance de l'autocollant par IA :
 - Utilisation d'un modèle **YOLOv8** (En version « Small ») qui est le compromis entre une détection précise et exécution rapide en local sans saturer la RAM.
 - Entièrement entraîné sur notre propre jeu de données que nous constitueront et qui seront annotées sur le site **Roboflow**.



Etude et Choix - Traitement OpenCV

- Il faut ensuite pouvoir manipuler l'image pour y incruster notre machine.
- Pour cela on utilise **OpenCV** :
 - Une des bibliothèques les plus performante pour les opérations matricielles lourdes.
 - Permet de calculer la déformation de perspective.
 - Gère le blending alpha pour incruster proprement l'appareil.
 - Peut créer des ombres portées et adapter la colorimétrie de l'équipement.
- Agit comme le véritable moteur de rendu de l'application.



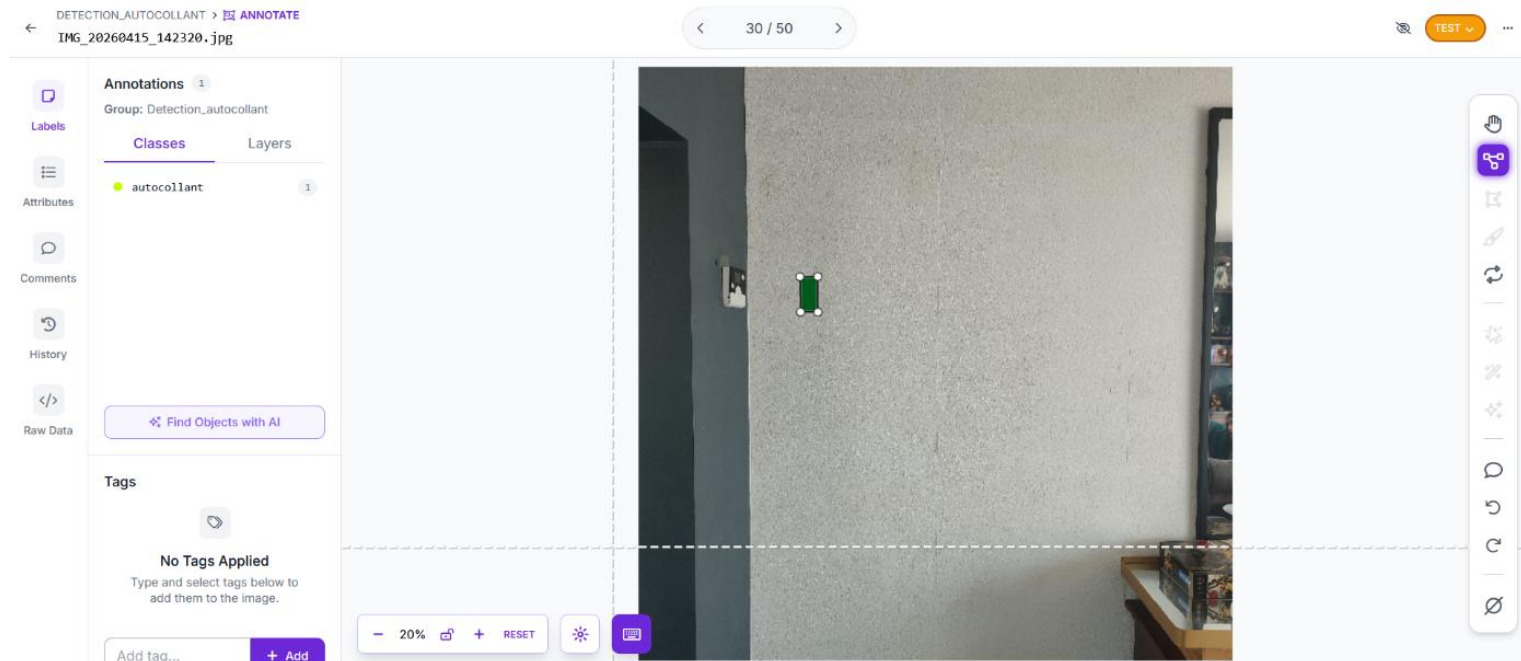
Etude et Choix - Flutter

- Pour faire l'application mobile, on avait besoin d'une interface fluide, robuste et facile à créer.
- Pour cela nous avons choisis d'utiliser **Flutter** :
 - Langage Dart très proche du C++ qu'on maîtrise bien.
 - Permet de développer une seule fois et de compiler pour Android et iOS, Flutter s'occupe de tout.
 - Permet de séparer les calculs lourds d'OpenCV de l'interface graphique pour éviter que l'écran ne fige.
 - Facilité de structuration du code.



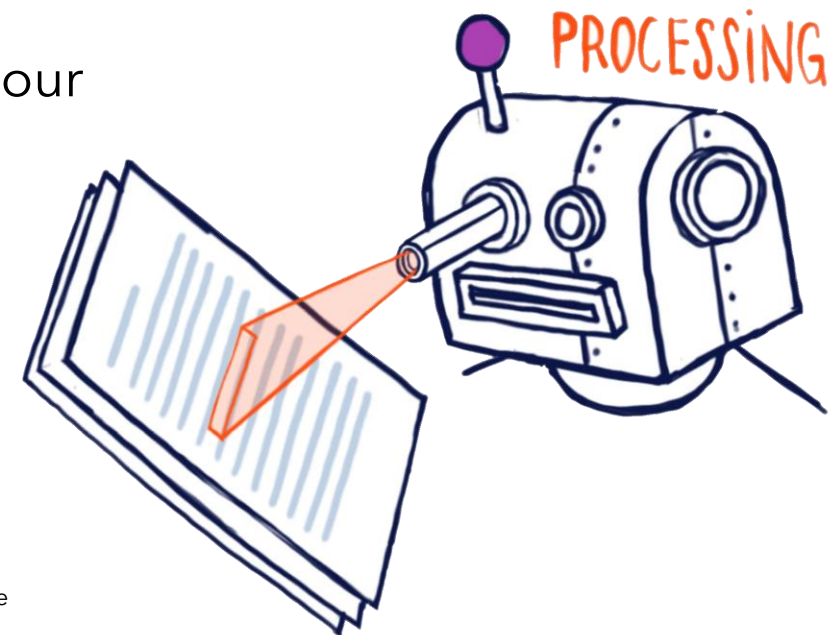
Implémentation – IA de Détection

- Pour que le modèle comprenne ce qu'est notre autocollant, on a créé notre propre dataset avec une prise de plus d'une cinquantaine de photos de l'autocollant sur différents types de murs et d'ambiance.
- Utilisation de Roboflow pour annoter les images en y indiquant la boîte englobante où se trouve l'autocollant et les quatre coins de l'autocollant. On y fait aussi de la data augmentation (Qui augmente le dataset à plus d'une centaine d'image).



Implémentation – IA de Détection

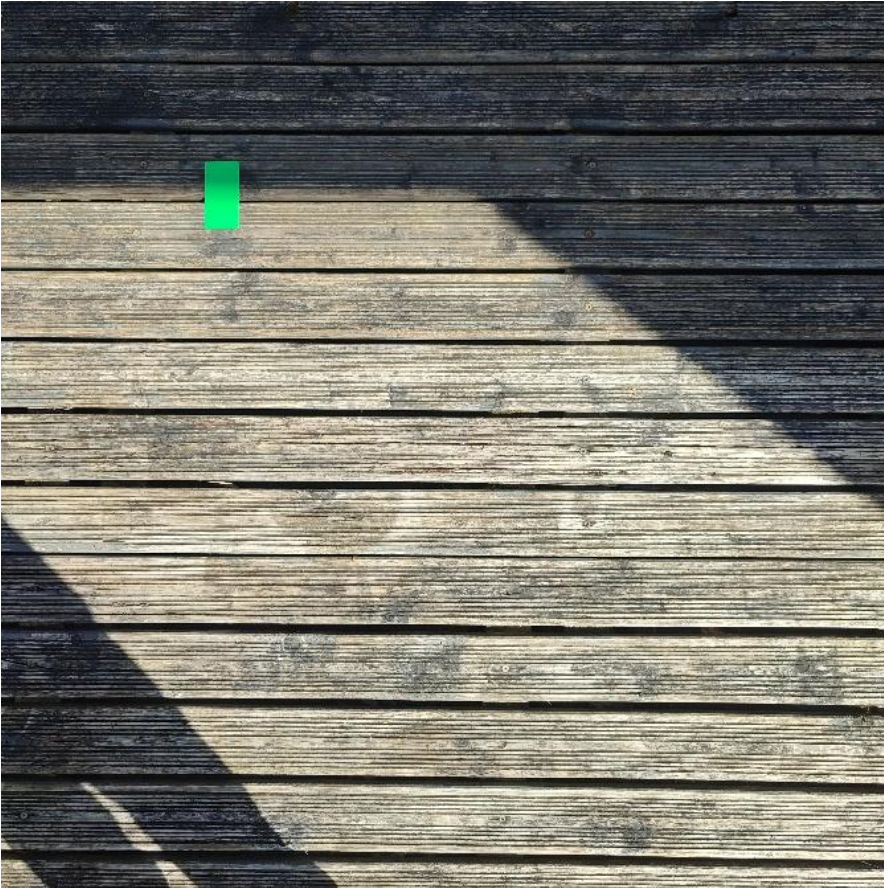
- Entraînement fait en Python puis exportation au format .tflite pour tourner sur mobile.
- Prétraitement et post-traitement des images sur le smartphone :
 - L'image est redimensionnée en 1024x1024 pixels et normalisée.
 - Tri mathématique strict des 4 points reçus de l'IA (haut-gauche, haut-droit, bas-droit, bas-gauche) pour éviter d'avoir un équipement tordu.
- Si l'IA n'est pas sûre à plus de 92%, l'application bascule sur les coins de la boîte englobante ou propose un placement manuel.



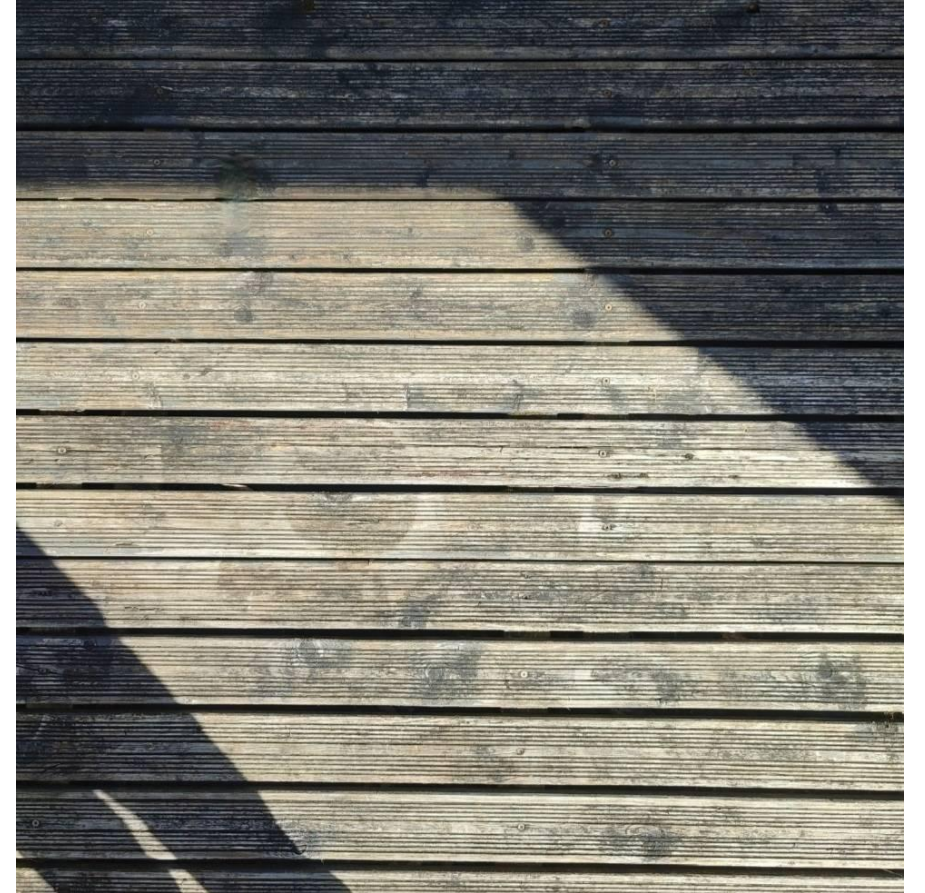
Implémentation – Inpainting

- La fonction **cv.inpaint** n'existant pas sur le package Flutter et même avec une méthode alternative on avait un résultat très peu convainquant pour cacher l'autocollant, on a donc dû faire autrement :
 - Intégration du modèle génératif **LaMa** (Large Mask Inpainting) qui tourne en local.
- Pour éviter de saturer la RAM du téléphone avec une image géante :
 - On coupe un carré de 2.5 fois la taille de l'autocollant et on réduit ce petit patch en 512x512 pixels avant de l'envoyer à l'IA qui générera un inpainting.
- Traitement pour que la disparition soit invisible à l'œil nu :
 - Avant inpainting, on remplit la zone de l'autocollant avec la couleur moyenne du mur.
 - Après traitement, on agrandit le patch via une interpolation et on applique un filtre de netteté.
 - Puis à la fin on injecte un léger bruit gaussien et on fait une fusion qui adoucie les bords.

Implémentation - Inpainting



Avant/Après
Inpainting



Implémentation – Traitement OpenCV

- Pour que l'équipement ressemble à un vrai produit physique et pas à un simple autocollant collé au mur, on applique ces traitements :
 - Déformation de perspective calculée à partir des 4 points de l'IA.
 - « Modélisation 3D » par extrusion du contour en fonction de la profondeur réelle.
 - Calcul d'un point de fuite central calqué sur l'appareil photo pour respecter la distorsion de l'objectif.
 - Adaptation automatique à l'éclairage de la pièce :
 - Filtres de Sobel pour orienter les ombres portées selon la vraie lumière du mur.
 - Passage en HSV pour que l'appareil absorbe la teinte ambiante et s'assombrisse si la zone est sombre.

Implémentation - Traitement OpenCV



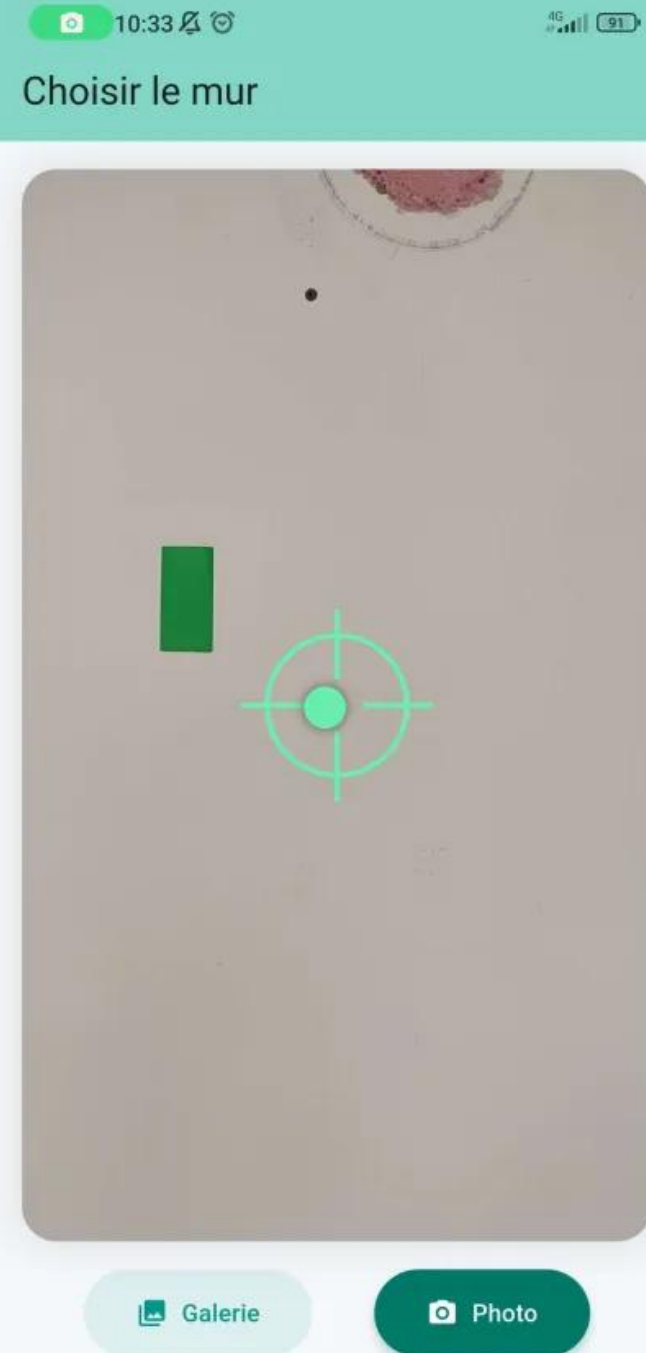
Première Incrustation d'une clim
au début du stage



Incrustation actuelle
d'une unité extérieure

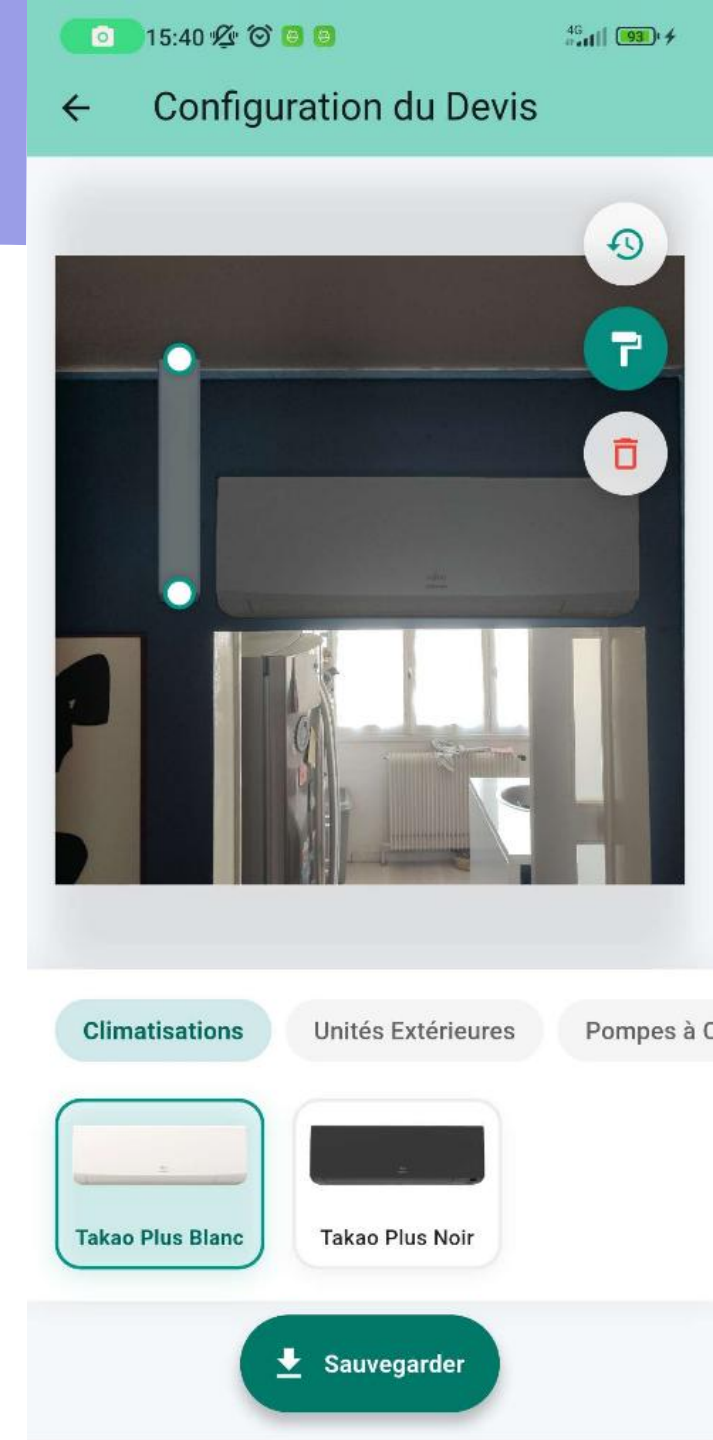
Implémentation – UI Flutter

- Nous avons développé l'interface utilisateur dont voici l'écran d'accueil où l'on assiste l'artisan pour prendre la meilleure photo possible :
 - Niveau à bulle gyroscopique : Utilise l'accéléromètre pour s'assurer que l'artisan est bien de face par rapport au mur. Le viseur passe au vert uniquement quand le smartphone est parfaitement droit.
 - Des performances fluides dès l'allumage pour pas que ça rame avec un préchargement des modèles d'IA en tâche de fond pour éviter les latences au moment de prendre la photo et des « isolate » dédié en arrière-plan pour compresser et optimiser l'image dès sa capture sans figer l'interface.



Implémentation – UI Flutter

- Et voici l'écran de traitement :
 - Déplacement fluide des appareils au doigt (Avec bouton annulation de celle-ci) et catalogue accessible en bas dans des catégories pour se repérer.
 - Possibilité de zoomer sur l'image et de sauvegarder une photo de l'incrustation.
 - Tracé des goulottes en plastique sur un mode accessible via un bouton. Traçage en dégradé et bloqué à 90° pour faire propre avec une loupe de précision dynamique au-dessus du doigt pour ajuster les goulottes au millimètre.
 - Slider de comparaison pour balayer l'écran et voir le mur « Avant / Après » en temps réel.



Perspective d'Amélioration

OpenCV

1. Optimiser les calculs lourds pour qu'ils se fassent plus vite.
2. Mieux gérer le premier plan en ajoutant un outil de gomme pour effacer l'équipement et faire passer des plantes, meubles, ect... par dessus

IA de Détection

1. Amélioration de la détection de l'autocollant lors des contre-jours.
2. Amélioration globale du dataset en cas d'autre galère de détection.
3. S'affranchir de l'autocollant si possible

L'Application

1. Elargir le catalogue d'équipement pour qu'il couvre tout ce que l'entreprise peut poser.
2. Permettre le traçage de goulotte dans plusieurs directions avec des coudes.

Conclusion

- En conclusion, ce stage a été une bonne occasion de rendre un projet complexe d'IA et de traitement d'image concret et utilisable par des artisans sur le terrain.
- Nous avons pu approfondir nos connaissances en développement mobile avec le Framework Flutter et en traitement d'images avec OpenCV.
- Cela nous a aussi permis de nous rendre compte des besoins des utilisateurs et que si l'application doit paraître simple pour l'artisan devant son client, elle demande beaucoup d'optimisations en arrière-plan pour nous développeur.
- Cette expérience fut très enrichissante sur le plan technique et humain et a renforcé mon désir de travailler dans le développement logiciel et l'IA.

**Merci de votre
Attention**

